Practical 5

1.

AIM :

Shape areas: an abstract Shape with abstract area(); subclasses Circle, Rectangle,

Triangle. Put a mix in a Shape[] and loop once to print each area and the running total,

tracking the largest — one loop handling every shape through polymorphism.

CODE :

```
abstract class Media {
    String title;
    int daysLate;

    Media(String title, int daysLate) {
        this.title = title;
        this.daysLate = daysLate;
    }

    abstract double lateFee();
}
```

```
class Book extends Media {
    Book(String title, int daysLate) {
super(title, daysLate);
    }

    double lateFee() {
return daysLate * 2;
    }
}

class Movie extends Media {
    Movie(String title, int daysLate) {
super(title, daysLate);
    }

    double lateFee() {
return daysLate * 5;
    }
}

class Game extends Media {
    Game(String title, int daysLate) {
super(title, daysLate);
```

```
    }
    double lateFee() {
return daysLate * 8;
    }
}

public class mediaFees {
    public static void main(String[] args) {
Media[] items = {           new
Book("Java Basics", 3),           new
Movie("Avengers", 2),           new
Game("Minecraft", 4),           new
Book("Data Structures", 5)
        };

        double total = 0;

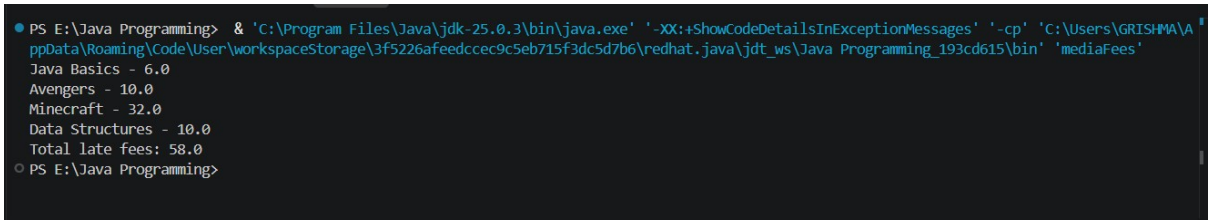
        for (Media m : items) {
double fee = m.lateFee();

            System.out.println(m.title + " - " + fee);

            total += fee;
```

```
        }
        System.out.println("Total late fees: " + total);
    }
}
```

OUTPUT :



2.

AIM :

Payroll: an abstract Employee with abstract monthlySalary(); FullTime (fixed), PartTime

(hours × rate) and Intern (stipend), each calling super for the shared name/id. Loop a

mixed array to print each salary and the total; use

instanceof to add a note only for interns.

CODE :

```
abstract class Employee {
String name;
    int id;

    Employee(String name, int id) {
this.name = name;
```

```
        this.id = id;
    }

    abstract double monthlySalary();
}

class FullTime extends Employee {
    double salary;

    FullTime(String name, int id, double salary) {
super(name, id);         this.salary = salary;
    }

    double monthlySalary() {
        return salary;
    }
}

class PartTime extends Employee {
    int hours;
double rate;
```

```
    PartTime(String name, int id, int hours, double rate) {
super(name, id);        this.hours = hours;        this.rate = rate;
  }

  double monthlySalary() {
    return hours * rate;
  }
}

class Intern extends Employee {
double stipend;

  Intern(String name, int id, double stipend) {
super(name, id);        this.stipend = stipend;
  }

  double monthlySalary() {
    return stipend;
  }
}

public class payroll {
```

```
    public static void main(String[] args) {
Employee[] employees = {            new
FullTime("Rahul", 101, 50000),            new
PartTime("Aman", 102, 80, 300),            new
Intern("Riya", 103, 12000),            new
FullTime("Neha", 104, 45000)
        };

        double total = 0;

        for (Employee e : employees) {
double salary = e.monthlySalary();
System.out.println(e.name + " - " + salary);

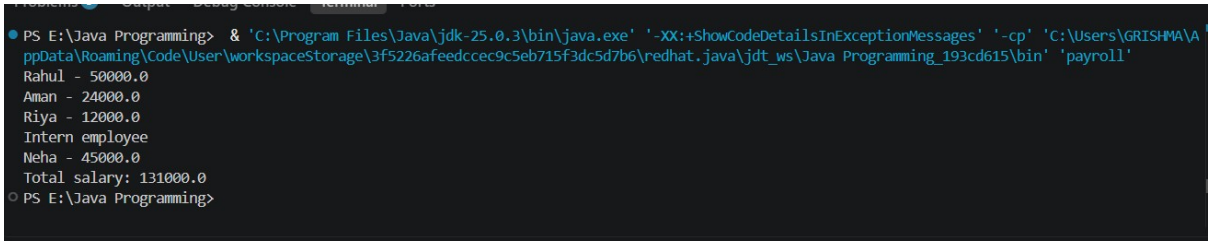
            if (e instanceof Intern) {
                System.out.println("Intern employee");
            }

            total += salary;
        }

        System.out.println("Total salary: " + total);
```

```
    }
}
```

OUTPUT :



3.

AIM :

(For Advanced Learners – home) Model a few media types that each compute a late fee

differently and total the fees for a returned batch. Design the hierarchy and rules yourself.

CODE :

```
abstract class Shape {
abstract double area();
}

class Circle extends Shape {
    double r;

    Circle(double r) {
        this.r = r;
    }
```

```
    double area() {
return Math.PI * r * r;
    }
}

class Rectangle extends Shape {
    double l, b;

    Rectangle(double l, double b) {
        this.l = l;
this.b = b;
    }

    double area() {
        return l * b;
    }
}

class Triangle extends Shape {
    double b, h;

    Triangle(double b, double h) {
```

```
        this.b = b;
this.h = h;     }

    double area() {
return 0.5 * b * h;
    }
}

public class shapeAreas {
    public static void main(String[] args) {
Shape[] shapes = {              new
Circle(5),              new Rectangle(4, 6),
new Triangle(8, 3),             new
Circle(3)
        };

        double total = 0;
double largest = 0;

        for (Shape s : shapes) {
double a = s.area();
System.out.println("Area: " + a);
```

```
            total += a;
            if (a > largest) {
largest = a;
            }
        }

        System.out.println("Total area: " + total);
        System.out.println("Largest area: " + largest);
    }
}
```

OUTPUT :



LEARNING OUTCOME :

1. Shape Areas

- Understand **abstract classes and abstract methods**.
- Apply **inheritance and method overriding**.

- Understand **polymorphism** using a Shape[] array.
- Calculate and compare areas using a single loop.

2. Payroll

- Understand **abstract classes and inheritance**.
- Use **constructors and super()** for shared employee details.
- Apply **method overriding** for different salary calculations.
- Use **instanceof** to identify specific employee types.

3. Media Late Fees

- Design a **class hierarchy** for different media types.
- Apply **polymorphism and method overriding**.
- Implement different fee rules for different media.
- Calculate the **total late fees** using a common array and loop.